

Software Requirements Specification for Inventory Management System

Version 1.0

Prepared by

Role	Team Member(s)	E-mail
Business Analyst	Likhith Reddy	se23uari056@mahindrauniversity.edu.in
Frontend Developer	Chaithanya Naidu	se23uari009@mahindrauniversity.edu.in
	Thilak Kusampudi	se23ucse099@mahindrauniversity.edu.in
Backend Developer	Sohan Varma	se23ucse163@mahindrauniversity.edu.in
	Rihanth	se23uari063@mahindrauniversity.edu.in
System Architect	Vishwa Tej Goud	se23uari072@mahindrauniversity.edu.in
Database Designer	Sindhu Reddy	se23ucse091@mahindrauniversity.edu.in
Testing	Punith	se23uari045@mahindrauniversity.edu.in

Client Contact	Software Engineering course, Mahindra University
Course	Software Engineering
Date	12 March 2026

Contents

1	Introduction
1.1	Document Purpose
1.2	Product Scope
1.3	Intended Audience and Document Overview
1.4	Definitions, Acronyms and Abbreviations
1.5	Document Conventions
1.6	References and Acknowledgments
2	Overall Description
2.1	Product Overview
2.2	Product Functionality
2.3	Design and Implementation Constraints
2.4	Assumptions and Dependencies
3	Specific Requirements
3.1	External Interface Requirements
3.2	Functional Requirements
3.3	Use Case Model
4	Other Non-functional Requirements
4.1	Performance Requirements
4.2	Safety and Security Requirements
4.3	Software Quality Attributes
5	Other Requirements
	Appendix A – Data Dictionary
	Appendix B – Group Log

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Prepared by our team based on project requirements with minor AI support.	Repackaged the existing SRS content into the IEEE-style template structure	12 March 2026

Introduction

This Software Requirements Specification (SRS) defines the functional, non-functional, and environmental requirements for an Inventory Management System intended for small businesses. The document reorganizes the provided requirements into an IEEE-style structure so the system can be designed, implemented, tested, and reviewed in a consistent way.

The remaining subsections describe the purpose and scope of the system, its interfaces, detailed requirements, quality attributes, and supporting appendices such as a data dictionary and group log.

1.1 Document Purpose

The purpose of this document is to provide a complete and traceable requirements baseline for the Inventory Management System, version 1.0. It captures what the system must do, the constraints under which it must operate, and the quality expectations that will be used to validate the final product.

This SRS is intended to support design, development, demonstration, and assessment activities. It is especially useful for aligning student developers, instructors, evaluators, and future maintainers around a common understanding of the system.

1.2 Product Scope

The Inventory Management System is a desktop- or web-based software solution that helps small businesses manage product catalogs, inventory counts, restocking, sales records, low-stock alerts, and basic reporting. The software is intended to reduce manual errors, provide up-to-date stock visibility, and improve day-to-day operational efficiency.

Key benefits include faster stock updates, better product traceability, support for barcode-driven workflows, and improved decision-making through dashboards and inventory reports. The system is designed to be simple enough for non-technical staff while remaining scalable to thousands of products.

1.3 Intended Audience and Document Overview

This document is intended for project stakeholders including developers, testers, instructors, project reviewers, and client representatives. Developers should focus on Sections 2, 3, and 4 for system behavior and constraints. Testers can use Sections 3 and 4 to derive test cases. Instructors and reviewers can use the complete document to assess whether the proposed system addresses the required scope.

The document is organized from a high-level context to detailed requirements. Readers should begin with the Introduction and Overall Description, then move to Specific Requirements and Other Non-functional Requirements, and finally review the appendices for data definitions and project-log support.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
API	Application Programming Interface
DBMS	Database Management System
GUI	Graphical User Interface
IMS	Inventory Management System

LTS	Long-Term Support
OS	Operating System
SRS	Software Requirements Specification
UI	User Interface

1.5 Document Conventions

This SRS follows a structured IEEE-style format. Section headings and subsection numbering are used to create a clear information hierarchy. Requirement statements use “shall” to express mandatory behavior. Tables are used to present environmental, unsupported, and quality-related requirements in a readable form.

Body text uses a consistent Arial-based layout with single-spaced paragraphs and 1-inch page margins. Functional requirements are labeled F1, F2, and so on, while use cases are labeled U1, U2, and U3 for traceability.

1.6 References and Acknowledgments

- Provided source document: “Inventory Management System – System Requirements.”
- Provided template: “IEEE Software Requirements Specification Template.”
- General IEEE SRS structuring conventions are used as the basis for document organization.

Overall Description

2.1 Product Overview

The Inventory Management System is a new, self-contained software product intended to support inventory operations for small businesses. It is designed to centralize product information, stock levels, transaction history, and user access in one maintainable system. The application may be deployed locally or in the cloud, depending on business needs.

At a high level, the system is used by administrators and staff members, exchanges data with suppliers during restocking workflows, and persists in all operational data in a relational database. The following context diagram illustrates the product’s operating environment.

Inventory Management System - Context View

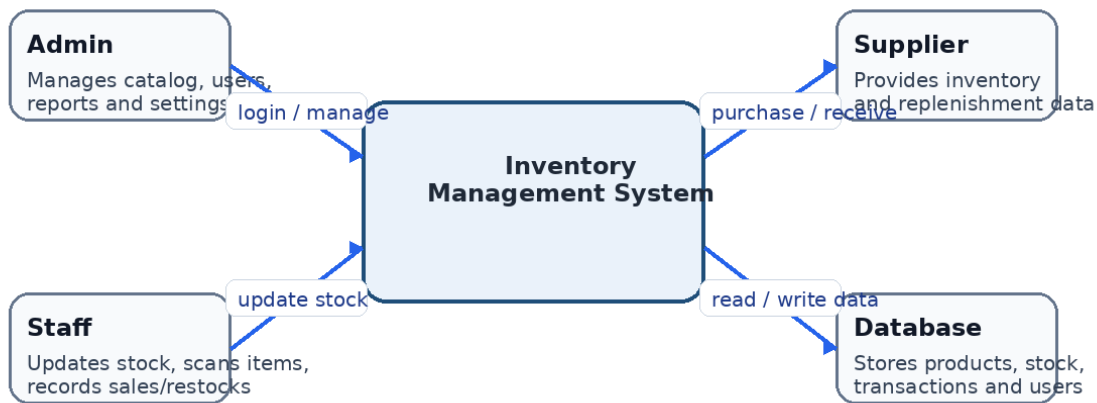


Figure 1. High-level context diagram for the Inventory Management System.

2.2 Product Functionality

- Authenticate users and enforce role-based access for Admin and Staff users.
- Create, edit, search, and organize product records.
- Track stock quantities and reflect updates immediately after sales, restocks, or corrections.
- Support barcode-assisted item lookup and scanning workflows.
- Record sales and inventory receipts as transactions.
- Generate low-stock alerts and basic operational reports.
- Store product, user, and transaction data in a relational database.
- Operate on common desktop platforms through a browser-based or GUI-based interface.

2.3 Design and Implementation Constraints

The design and implementation of the Inventory Management System are constrained by the system environment, hardware assumptions, runtime stack, and deployment expectations supplied in the original requirements. These constraints are summarized below.

Category	Requirement Type	Minimum Requirement	Recommended Requirement	Notes
Operating System	Desktop OS	Windows 10 / Linux / macOS	Latest stable OS	Must support web or GUI applications
Platform Type	Device	Desktop / Laptop	Modern desktop/laptop	Primary usage device for small businesses

Runtime Environment	Server Runtime	Node.js / Python runtime	Latest LTS version	Required for backend services
Database	DB System	MySQL / PostgreSQL	Latest stable version	Stores inventory, products, and transaction data
Internet Connectivity	Network	Active internet connection	Stable broadband	Needed for analytics and updates
RAM	Memory	4 GB	8 GB+	Ensures smooth dashboard analytics
Storage	Disk Space	500 MB free	1 GB+ free	Stores application data and logs
CPU	Processor	Dual-core processor	Quad-core modern CPU	Improves report generation speed
Browser	Web Browser	Chrome / Firefox	Latest browser versions	Required for web dashboard
Barcode Scanner	Peripheral	Basic USB barcode scanner	Mobile barcode scanner	Used for product scanning
Keyboard	Peripheral	Standard keyboard	—	Required for data entry
Mouse	Peripheral	Standard mouse	—	Required for UI navigation
Permissions	System Access	User install permissions	Admin rights recommended	Needed for setup and updates
Execution Capability	Runtime	Local machine or server	Cloud deployment	Supports scalability
Environment Type	Deployment	Local server	Cloud hosting	Optional cloud deployment

Explicitly not supported environments are shown below.

Unsupported Environment	Reason
Mobile-only systems	System designed primarily for desktop dashboard usage
Offline-only environment	Analytics and updates require internet
Very low-end hardware	May not support analytics and reporting modules

2.4 Assumptions and Dependencies

- A relational database such as MySQL or PostgreSQL will be available and correctly configured.
- Users will access the system from a desktop or laptop device rather than a mobile-only environment.
- Internet connectivity is available whenever analytics, updates, or cloud-hosted services are needed.
- A barcode scanner may be integrated as an optional productivity aid, but the system shall still support manual entry.
- Only two main operational roles are assumed in this version of the system: Admin and Staff.

Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system shall provide a simple dashboard-oriented user interface suitable for non-technical users. The primary screens shall include login, product list, product detail, stock update, transaction history, alerts, and reports. Users shall interact with the system through a standard keyboard and mouse, and optionally through barcode scans for product lookup.

3.1.2 Hardware Interfaces

- Desktop or laptop computer
- Keyboard
- Mouse
- Optional USB or mobile barcode scanner
- Server or host machine for backend runtime and database services

3.1.3 Software Interfaces

- Backend runtime based on Node.js or Python
- Relational database interface for MySQL or PostgreSQL
- Browser interface compatible with Chrome and Firefox
- Optional analytics or update services requiring internet access

3.2 Functional Requirements

F1: The system shall authenticate users before allowing access to protected functions.

F2: The system shall support role-based access with at least Admin and Staff roles.

F3: The system shall allow authorized users to add, edit, and view product records.

F4: The system shall maintain the current stock quantity for each product.

F5: The system shall update inventory levels immediately when a sale, restock, or correction is recorded.

F6: The system shall allow product lookup by name, SKU, or barcode value.

F7: The system shall generate low-stock alerts when inventory falls below a defined threshold.

F8: The system shall store product, stock, user, and transaction data in a persistent relational database.

F9: The system shall generate basic reports for inventory status and transaction activity.

F10: The system shall validate user input and display meaningful error messages when invalid values are entered.

3.3 Use Case Model

The primary actors are Admin, Staff, and Supplier. The use case model below summarizes the main interactions with the system.

Use Case Model

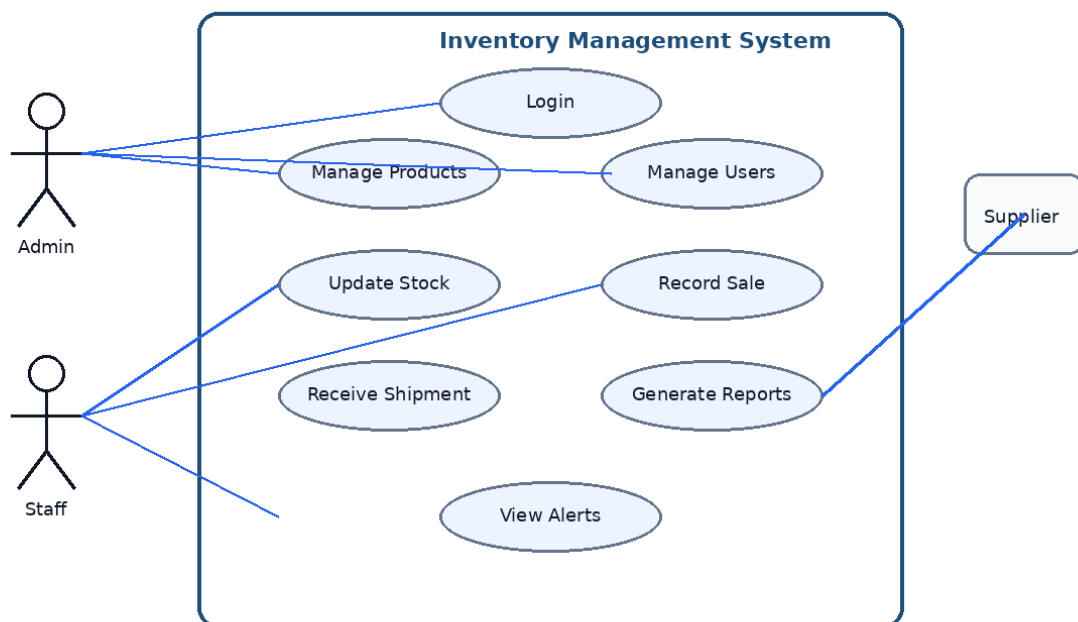


Figure 2. Use case model for the Inventory Management System.

U1 – Manage Products

Author: Team

Purpose: Create, update, search, or review product information.

Requirements Traceability: F2, F3, F6, F8

Priority: High

Preconditions: User is authenticated and authorized.

Postconditions: Product data is stored or updated in the database.

Actors: Admin, Staff

Extends: None

Basic Flow: 1. User opens product module. 2. User searches or selects a product. 3. User creates or edits product details. 4. System validates input. 5. System saves the data and confirms success.

Alternative Flow: If barcode input is available, the user scans an item to retrieve the product directly.

Exceptions: System rejects duplicate or invalid product values and displays an error message.

Includes: Login (U0 implied)

Notes/Issues: Product identifiers should remain unique.

U2 – Update Stock / Record Sale

Author: Team

Purpose: Adjust stock quantities after sales, restock, or corrections.

Requirements Traceability: F4, F5, F8, F10

Priority: High

Preconditions: User is authenticated, and a valid product exists.

Postconditions: Inventory quantity and transaction history are updated.

Actors: Admin, Staff

Extends: None

Basic Flow: 1. User opens stock updates or sales screens.

2. The user selects a product.

3. User enters quantity change.

4. System validates entry.

5. System updates stock and writes a transaction record.

Alternative Flow: A barcode scan may populate the product automatically.

Exceptions: Negative or invalid values are rejected with a clear error message.

Includes: Login (U0 implied)

Notes/Issues: The update should be reflected immediately on the dashboard.

U3 – Generate Reports and Alerts

Author: Team

Purpose: Review inventory status, low-stock conditions, and transaction summaries.

Requirements Traceability: F7, F9

Priority: Medium

Preconditions: User is authenticated, and reporting data exists.

Postconditions: Requested report or alert view is displayed.

Actors: Admin

Extends: None

Basic Flow: 1. Admin opens a reporting module.

2. System loads current stock and transaction metrics.

3. Admin selects a report or alert view.

4. System displays the output.

Alternative Flow: Filters may be applied to narrow the reporting period or product range.

Exceptions: If no data exists, the system displays an empty-state message rather than failing.

Includes: Login (U0 implied)

Notes/Issues: Large catalogs should still be handled without excessive delay.

Other Non-functional Requirements

4.1 Performance Requirements

The system shall meet the following measurable performance expectations derived from the provided requirements.

Category	Requirement	Description	Expected Standard / Criteria
Performance	Response Time	System should respond quickly to user actions	Page loads within 2 seconds
Performance	Data Processing	Inventory updates processed instantly	Stock updates reflected immediately
Scalability	Inventory Size	System should handle large product catalogs	Support thousands of products
Reliability	Error Handling	System should show clear error messages	Application should not crash
Reliability	Data Recovery	Prevent loss of transaction data	Automatic database backups recommended
Usability	Ease of Use	Interface should be simple for non-technical users	Users learn system within minutes
Usability	User Feedback	Clear notifications for stock alerts	Visible alert messages
Maintainability	Code Structure	Modular and well-documented code	Easy feature updates
Portability	Cross Platform	Runs on Windows, Linux, macOS	Web-based compatibility
Security	Authentication	Secure login and role-based access	Admin and Staff roles enforced
Security	Data Protection	Prevent unauthorized access to data	Role permissions and validation
Availability	System Availability	System accessible during working hours	Minimal downtime
Robustness	Invalid Input Handling	Invalid product or stock values handled safely	Meaningful validation messages

Resource Usage	Memory Efficiency	System should not consume excessive memory	Typical usage below 2 GB
Resource Usage	CPU Usage	Moderate CPU usage during analytics	Should not overload CPU

4.2 Safety and Security Requirements

- The system shall require authenticated access before exposing inventory or transaction data.
- The system shall enforce role-based permissions so only authorized users can manage products, stock, users, and reports.
- The system shall validate all user-supplied data to prevent invalid stock values, malformed records, or accidental corruption of data.
- The system should protect against unauthorized access to database records through secure backend validation and controlled database connectivity.
- The system should support periodic backups or recovery procedures to reduce the risk of transaction data loss.

4.3 Software Quality Attributes

4.3.1 Reliability

The system shall remain stable during normal daily usage and shall present clear error messages instead of crashing. Inventory updates and transaction writes shall persist consistently so that stock counts remain trustworthy.

4.3.2 Usability

The interface shall be simple enough that non-technical users can learn the main workflow within minutes. Common tasks such as product search, stock update, and alert review shall require minimal steps and provide visible feedback.

4.3.3 Maintainability

The software shall be organized into modular components, so future updates, bug fixes, and new features can be implemented without major redesign. Backend services, database access, and UI logic should remain logically separated and documented.

4.3.4 Portability

The solution shall support deployment on major desktop operating systems through a standards-based browser or equivalent GUI environment. The design should also allow movement between local and cloud-hosted deployments.

Other Requirements

This system uses a relational data model and should include, at minimum, tables for Users, Products, Inventory, Transactions, and Suppliers. Product records should be uniquely identifiable, and transaction records should retain timestamps for auditability. If future versions add analytics or integrations, those features should build this base data model rather than replace it.

Appendix A – Data Dictionary

Item	Type	Description
UserID	Identifier	Unique identifier for a user account
Role	Enum	User role such as Admin or Staff
ProductID	Identifier	Unique identifier for a product
ProductName	Text	Human-readable product name
SKU	Text	Stock keeping unit or internal code
Barcode	Text	Scan value for product lookup
QuantityOnHand	Integer	Current available stock quantity
ReorderThreshold	Integer	Minimum stock level before alert
TransactionID	Identifier	Unique identifier for an inventory transaction
TransactionType	Enum	Sale, Restock, or Adjustment
TransactionQuantity	Integer	Quantity involved in a transaction
TransactionTimestamp	Datetime	Date and time of transaction
SupplierID	Identifier	Unique identifier for a supplier record

Appendix B – Group Log

This section is reserved for team meeting minutes, task allocation notes, progress summaries, and evidence of collaboration. Because the provided source document did not include group-log details, this appendix should be completed by the project team before final submission.

Date	Participants	Activity Summary	Outcome
To be updated	To be updated	Initial requirements review and template adaptation	Draft SRS created
To be updated	To be updated	Design discussion	Pending
To be updated	To be updated	Final review and submission preparation	Pending